

FULL STACK DEVELOPMENT

FASE 5 | AULA 07 -

CLEAN ARCHITECTURE - THE ARCHITECTURE

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	13
REFERÊNCIAS.....	14

EMSE

O QUE VEM POR AÍ?

Os repositórios de dados, o modo de acesso à aplicação e tudo o que se relaciona com o mundo exterior, no contexto da Arquitetura Limpa, são tratados como meros detalhes. E isso não quer dizer que não sejam importantes, mas sim que devem ser considerados com a devida cautela; porque o software não é mais dependente desses itens, e funciona bem independente destes recursos.

Se a Arquitetura Limpa consegue isolar as regras de negócio do resto da aplicação, um dos principais responsáveis é o conceito de tratar o mundo exterior como um grupo de “detalhes”.

HANDS ON

Vamos entender, com alguns exemplos, como pensar a arquitetura e como tomar boas decisões de arquitetura de acordo com as restrições impostas por alguns detalhes. Também discutiremos um pouco sobre arquiteturas limpas e como balancear a realidade com o desenho da arquitetura.

Para essa aula, temos um link para você poder treinar o conteúdo ensinado. Acesse: [Base de dados aula 07](#).



SAIBA MAIS

O que apresentamos, até este momento, nos guia no caminho do isolamento de responsabilidades e da injeção de dependência, fazendo com que o foco da aplicação seja em resolver os problemas que ela deve resolver, e interagir com os componentes externos de forma independente, ou seja, fazer com que o software ou a solução funcione da forma mais independente possível do ambiente em que está funcionando.

Isso nos mostra duas características da Arquitetura Limpa e que às vezes não ficam tão claras para quem não conhece muito sobre Arquitetura de Software: o **isolamento do contexto de regra de negócio do contexto de execução do software** e a **independência do software em relação ao contexto de execução**.

No primeiro item, o que fazemos é isolar os componentes das regras de negócio e que entregam as funcionalidades, o “quê” do software, dos componentes que fazem a comunicação com o ambiente de execução que será usado para entregar os resultados das funcionalidades para os usuários, o “como”.

Já no segundo item, o foco é fazer com que o software não seja **diretamente** dependente de um tipo específico de Repositório de Dados ou de um tipo de arquitetura de comunicação, como uma API web, e do framework de implementação que este usa. As funcionalidades do software não são estruturadas de acordo com um framework.

Se antes o desenho do software era algo que se parecia com o que vemos na figura 1 – “Monólito”, onde as regras de negócio estão espalhadas por diversos componentes, e até mesmo servindo como a integração entre estes componentes, no desenho proposto na Arquitetura Limpa, que pode ser visto na figura 2 – “Componentes isolados”, nosso desenho se parece com algo mais organizado.

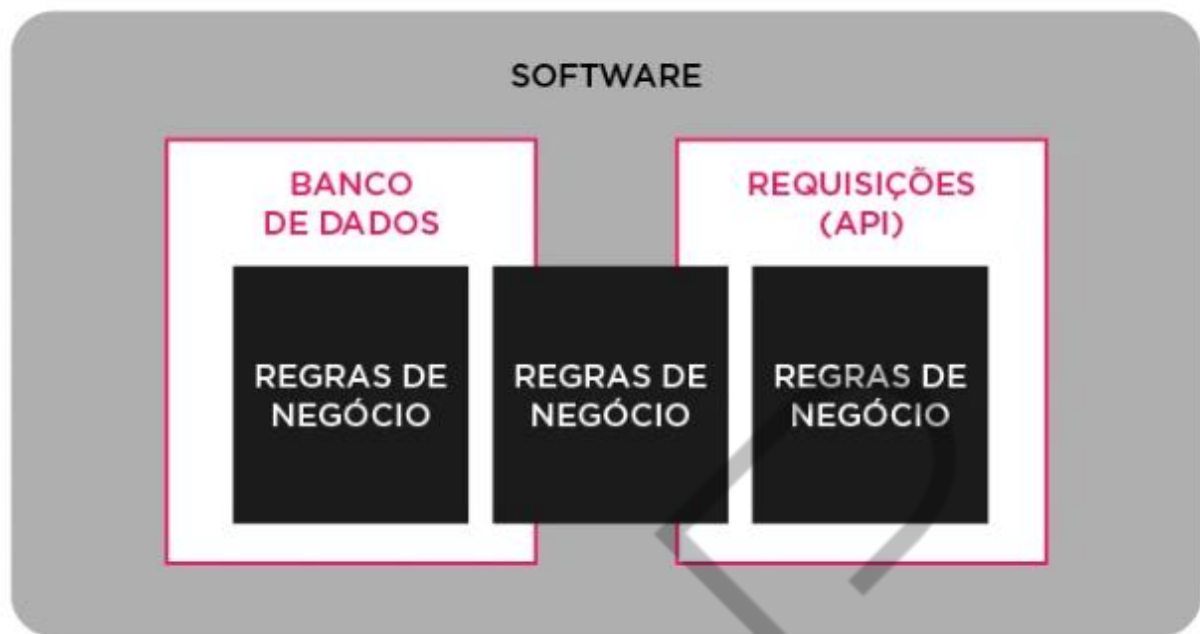


Figura 1 - Monólito
Fonte: elaborado pelo autor (2023), adaptado por FIAP (2023)

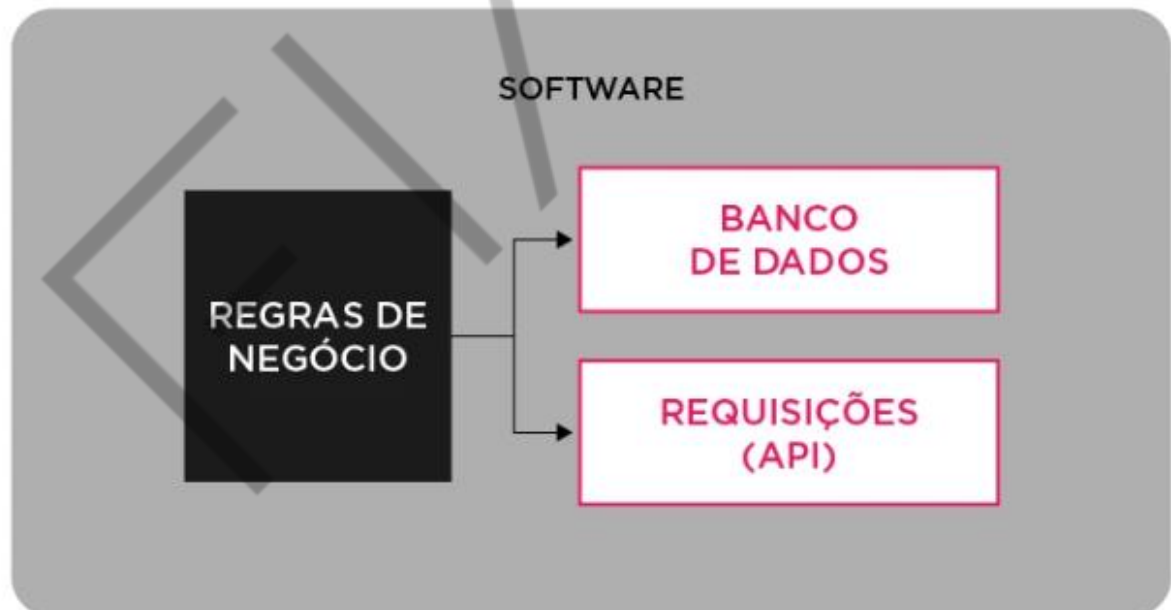


Figura 2 - Componentes Isolados
Fonte: elaborado pelo autor (2023), adaptado por FIAP (2023)

A mudança da estrutura das arquiteturas

Isso está relacionado a um ponto importante que foi levantado nos últimos anos no panorama da construção de software.

Anteriormente, o ambiente de execução e utilização do software era homogêneo. O software rodava dentro de uma empresa, em um ambiente definido pela disponibilidade de recursos já disponíveis ou em processo de aquisição. O projeto de software e, conseqüentemente a arquitetura desse software, eram restritos a este contexto: um único banco de dados, um ambiente de execução com rede isolada e computadores de clientes bem conhecidos, muitas vezes rodando um software nativo instalado diretamente no computador da pessoa que o utilizava. A necessidade de integração era quase inexistente, e quando acontecia era, muitas vezes, baseada na troca de dados de forma desconectada e assíncrona.

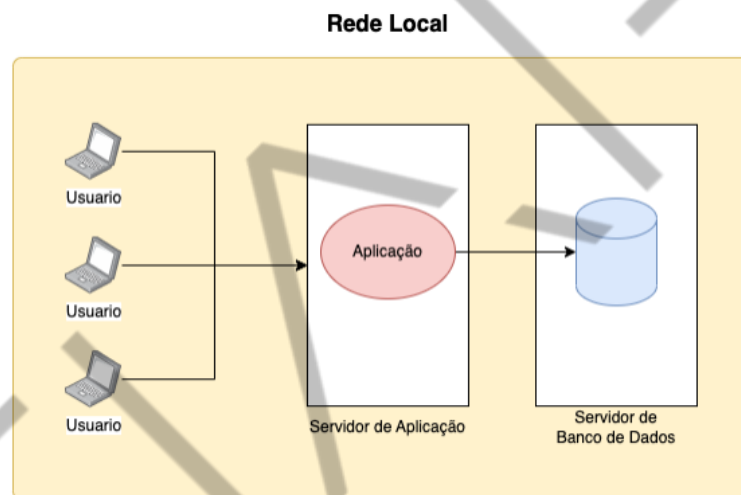


Figura 3 - Arquitetura de solução pré-internet
Fonte: elaborado pelo autor (2023), adaptado por FIAP (2023)

Até mesmo em grandes empresas, a realidade era semelhante: os data centers internalizados, as conexões entre filiais de forma precária ou restritas por definição. A interconexão entre os dados e os softwares eram restritos a ambientes internos.

Mas, nos últimos anos, o panorama de construção de software mudou e muito; a possibilidade de se conectar a diferentes computadores em ambientes desconhecidos, a possibilidade de acesso ao software por qualquer dispositivo, a disponibilização de ambientes de infraestrutura em nuvem e baseada em serviços, e o aumento de oferta de soluções diferentes para resolver os diferentes problemas mudaram o modo como pensamos a arquitetura, seja de software ou de solução.

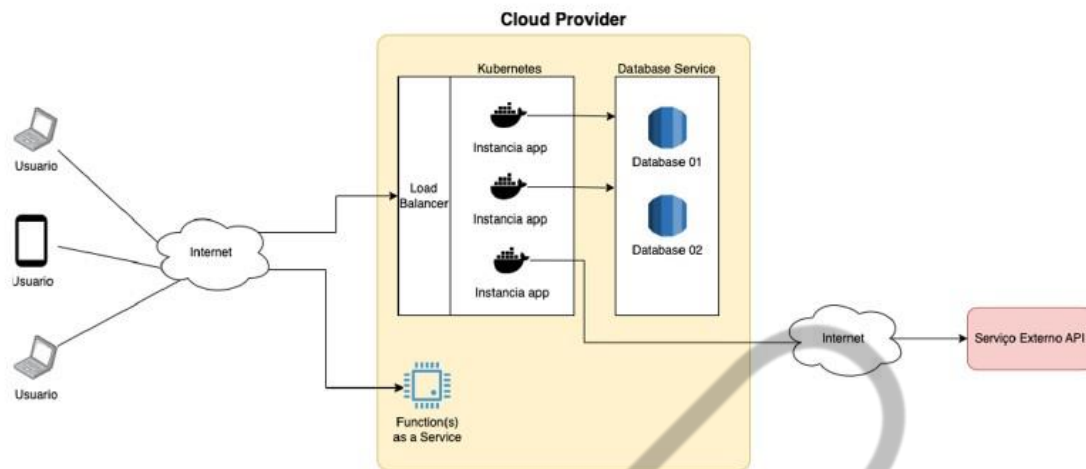


Figura 4 - Arquitetura de uma solução atual
Fonte: elaborado pelo autor (2023), adaptado por FIAP (2023)

Na arquitetura exemplificada na figura 4 – “Arquitetura de uma solução atual”, vemos que a solução é composta por componentes que se conectam a diferentes serviços, que recebem requisições de diferentes dispositivos muitas vezes desconhecidos, mas que seguem alguns padrões conhecidos (dispositivos móveis Android e iOS e navegadores). O desafio hoje é ter um software ou uma solução, composta de diferentes componentes ou softwares, que consigam resolver problemas dentro desse contexto heterogêneo.

Os repositórios de dados

Os dados utilizados pelo software estão disponíveis em um recurso fora do escopo do software, e é usado como um serviço. Até por isso, nos referimos cada vez mais a estes recursos como “repositórios de dados”, e não mais como simples “bancos de dados”, porque hoje temos diferentes soluções que servem como origens e destinos de dados.

- A comunicação com uma API externa pode ser tratada como uma origem de dados válida, e isolada como um componente de acesso a dados.

- Existem diferentes tecnologias de bancos de dados que podem ser utilizadas pelo software, de acordo com variáveis como desempenho e tipo de armazenamento de dados.

Com isso em mente, podemos considerar estes componentes tão diferentes como detalhes externos à nossa implementação. Porque para as nossas camadas internas, não importa a origem dos dados ou o destino, e sim que estes funcionem como o esperado quando necessário. Por isso implementamos, na camada mais externa, os componentes que fazem a comunicação com estes repositórios, e criamos Gateways que padronizam a comunicação entre nossas funcionalidades e regras de negócio com o mundo exterior.

Trabalhamos apenas com os dados, requisitando e enviando para os componentes que cuidam da comunicação. Porque nos importa, internamente, o uso dos dados, e não a sua origem. Essa origem é apenas um detalhe.

O acesso à aplicação

Precisamos pensar o nosso software para ser acessado por diferentes clientes, sejam nos dispositivos móveis, navegadores, desktop, entre outros. E ainda devemos estar preparados(as), para que o software seja resiliente, funcione bem em qualquer tipo de integração e que esteja pronto para ser adaptado rapidamente para uso por outros(as) clientes que não foram identificados(as) no início do projeto.

Usualmente, esse tipo de acesso se dá por meio de uma API, e o tipo mais famoso de API que existe é a acessível via protocolo HTTP, ou seja, as WEB APIs. Mas este não é o único tipo de **porta de entrada** que o nosso software pode disponibilizar, porque outros tipos de comunicação existem em diferentes ambientes: acesso via socket, via cliente nativo(a) ou até mesmo interfaces para comunicação por eventos.

Se as regras de negócio ou até mesmo a camada de controle de execução de atividades do software, estiver acoplada a um destes componentes, criamos um problema: o software só pode ser acessado de uma maneira. A partir daí os problemas ficam distribuídos: uma aplicação que precisa comunicar com o software precisa, obrigatoriamente, implementar a comunicação no modelo existente, muitas vezes criando mais softwares apenas para fazer esta adaptação de forma não natural, ou

precisamos alterar o software para ser usado por outro tipo de cliente, e isso impacta em reescrever a camada de comunicação, muitas vezes duplicando a regra de negócio para os dois componentes ou segregando a regra de negócio de forma parcial, e usando essa regra pelos componentes. Esse isolamento acaba sendo circunstancial e muitas vezes injetando problemas inesperados no software.

Por isso, isolar o acesso à aplicação em um ou mais componentes isolados permite que a comunicação seja construída baseada nas capacidades que as camadas mais internas fornecem, e assim cada interface pode tratar de fazer aquilo que é sua responsabilidade, ou seja, garantir a comunicação entre o software e os clientes.

Internamente, trabalhamos apenas com as requisições recebidas pelos nossos Controllers, e trabalhamos para devolver, ao componente requisitante, o que ele precisa; como a comunicação funciona é apenas um **detalhe**.

Um exemplo: Backend for frontend

Um exemplo prático do que vimos acima é a segregação de chamadas, nas APIs, para os diferentes tipos de cliente, uma técnica que é conhecida como “Backend-for-Frontend”, ou seja, criar um grupo de APIs (Backend) para cada cliente que for acessar (Frontend).

Isso faz com que a responsabilidade do componente de API esteja segregada em relação ao seu uso, e consiga atender ao cliente de acordo com sua necessidade e suas características. Por exemplo, se um(a) cliente móvel e um(a) cliente desktop tentam acessar o software, a princípio podemos tratar a requisição a partir do mesmo endpoint, mesmo que um(a) dos(as) clientes precise de um retorno com menos dados. Mas isso pode causar problemas para o código que implementa a funcionalidade, já que ele precisa entender qual a característica da chamada e qual o formato do retorno.

Se isolamos isso na camada de API, criando um endpoint para cada chamada, o que fazemos é segregar as responsabilidades do atendimento a cada chamada diretamente na implementação, como exemplificado a seguir.

```
app.get('/mobile/products', (_req, _res) => {  
  const pc = new ProductsController();  
  const all_products = pc.getAllWithFields(['name', 'identifier']);  
  return JSON.stringify(all_products);  
})  
  
app.get('/browser/products', (_req, _res) => {  
  const pc = new ProductsController();  
  const all_products = pc.getAll();  
  return JSON.stringify(all_products);  
})
```

No exemplo anterior, isolamos o acesso do cliente de acordo com sua necessidade na camada mais externa. Isso nos ajuda a manter a resposta segregada sem precisar tomar decisões dentro do código que não sejam relacionadas à entrega que o cliente necessita e nem integrar qualquer regra de negócio à camada de Controller para decidir como atender a um cliente ou outro.

Frameworks

Hoje o uso de frameworks está disseminado como uma ótima solução de apoio à criação de software. Seja o uso de frameworks full-stack (que tem componentes de várias camadas e responsabilidades) ou de micro-frameworks altamente especializados em um tipo de trabalho, usar uma estrutura base de código onde algumas das implementações triviais já estão prontas é importante para a entrega de software com mais qualidade e rapidez.

Mas como usar os frameworks na Arquitetura Limpa, sendo que o recomendado é sempre manter os componentes isolados, e muitos dos componentes de frameworks são integrados? Este desafio relembra um pouco a dificuldade de criar um software altamente acoplado a um tipo de serviço ou tecnologia.

Podemos adotar algumas abordagens: a primeira é fazer uso de micro-frameworks para construir o software. Assim, é possível construir os componentes com os frameworks e manter o acoplamento baixo, ou seja, cada componente ainda faz apenas uma tarefa e tem apenas uma responsabilidade.

A segunda abordagem é usar um framework mais robusto, e as ferramentas do framework a favor da estrutura do software, e não basear o código no que o framework

indica. Isso tende a ser um pouco mais trabalhoso, mas permite que a estrutura seja mantida. Ainda que o software ainda seja acoplado a um framework, conseguimos isolar bem as responsabilidades e camadas. Se o framework oferece uma camada de acesso a dados facilitada (como um ORM), podemos isolar todo o acesso a dados em uma camada, fazer com que ela seja utilizada pelos outros componentes, e que esta camada seja usada como um Gateway dentro dos Controllers. Outro ponto é isolar os casos de uso em relação aos Controllers, em uma biblioteca dentro do software. Com isso, conseguimos uma alta aderência às ideias da Arquitetura Limpa.

EXEMPLO

O QUE VOCÊ VIU NESTA AULA?

Vimos que não é possível entregar um software de qualidade se não usarmos os recursos externos para operacionalizar o trabalho da regra de negócio e as funcionalidades que entregamos. Mas isso não quer dizer, necessariamente, que precisamos depender diretamente e fortemente de um recurso externo para que o nosso software funcione corretamente. Ao contrário, devemos preparar o software para trabalhar bem com qualquer recurso que ele precise usar.

Para isso, vimos como pensar nesses recursos externos como detalhe, seja os recursos de obtenção e armazenamento de dados, ou os recursos que permitem que os clientes operem o software. Pensando nesses recursos como detalhes, construímos uma arquitetura robusta e à prova de alterações nos recursos que utiliza.

O que achou do conteúdo? Conte-nos no Discord! Estamos disponíveis para tirar dúvidas, fazer networking, enviar avisos e muito mais.

REFERÊNCIAS

EVANS, E. **Domain-Driven Design: Tackling Complexity in the Heart of Software**.

[s.l.]: Addison-Wesley Professional, 2003.

FOWLER, M. [s.d.]. Disponível em: <https://martinfowler.com/bliki/UseCase.html>.

Acesso em: 13 jun. 2023.

MARTIN, R. **Arquitetura Limpa**. Rio de Janeiro: Alta Books Editora, 2019.

MARTIN, R. **Clean Code: A Handbook of Agile Software Craftsmanship**. [s.l.]: Pearson, 2008.

EVANS

PALAVRAS-CHAVE

Repositório de dados. Estrutura de arquitetura. BFF.

EMSE

POSTECH